

Introduction

The goal of this lab is to approximate the periodic function

$$f(x) = \sin(2x) + \sin(x)$$

using a neural network (similar SineNet implementation from the lecture document).

Approximating periodic functions with neural models demonstrates their power as universal function approximators; here the network is trained to learn the mapping

$x \rightarrow f(x)$ from sampled data. Compared with a single sine, this function introduces higher frequency components and a more complex shape, which makes optimization and representation slightly more challenging.

I experimented with different activation functions, optimizers, dataset sizes (training on a larger superset and evaluating on the target set), and loss functions. The model was trained to minimize mean squared error (MSE) and evaluated both quantitatively (final loss) and qualitatively (predicted vs. true function plots). My best case SineNet achieved a loss of approximately 3×10^{-9} and closely matches the analytical function across the evaluation domain.

Task

Task of the lab as mentioned previously was to approximate the periodic function,

$$f(x) = \sin(2x) + \sin(x)$$

using a neural network. I implemented a custom model called SineNet, inspired by the lecture material but extended with additional modifications to improve flexibility and performance.

To optimize the network, I experimented with:

- **Dataset Sizes:** Training on larger datasets (2.5×, 5×, and 10× the original size) and evaluating on the original dataset.
- **Activation Functions:** Tested Tanh, SiLU, and a custom sin activation function.
- **Training Configuration:** AdamW optimizer, L2 loss function (MSE), learning rate scheduler (*ReduceLROnPlateau*), 1000 epochs.
- **Network Architecture:** 3 hidden layers (128 → 128 → 64 units).

Key findings during experimentation:

- Training on a dataset 5× larger than the evaluation set achieved the best balance between generalization and accuracy, while a 10× dataset did not provide meaningful improvement and risked overfitting.
- A sinusoidal activation function significantly outperformed both tanh and SiLU, which struggled to capture the periodicity of the target function.

- With the chosen configuration (AdamW, L2 loss, 1000 epochs, 5× dataset), the model achieved a final loss of approximately $1.6e-7$.

Code

Structure of the Model:

```
class SinActivation(nn.Module): # Made it
    def forward(self, x):
        return torch.sin(x)

class SineNet(nn.Module):
    def __init__(self):
        super(SineNet, self).__init__()
        self.hidden1 = nn.Linear(1, 128) #
        self.hidden2 = nn.Linear(128, 128)
        self.hidden3 = nn.Linear(128, 64)
        self.output = nn.Linear(64, 1)
        self.sin = SinActivation()
        # self.tanh = nn.Tanh() # Better t
        # self.silu = nn.SiLU() # Better t

    def forward(self, x):
        x = self.sin(self.hidden1(x))
        x = self.sin(self.hidden2(x))
        x = self.sin(self.hidden3(x))
        x = self.output(x)
        return x

# Instantiate the model (from class create
model = SineNet()
```

I based my implementation on the general structure from the lecture notes, but introduced several modifications to improve accuracy and stability. My model, SineNet, differs in the number of layers, neurons, and activation functions compared to the baseline.

Model Architecture

The final architecture of SineNet consists of:

- **Input layer:** 1 neuron
- **Hidden layers:** 128 → 128 → 64 neurons
- **Output layer:** 1 neuron
- **Activation:** custom sinusoidal activation function (SinActivation)

The intuition was that the function contains higher frequency components compared to a single sine, so the network required more expressive capacity. Increasing the number of layers and neurons boosted performance by ~40% compared to smaller setups.

Experiments

1. Number of Layers & Neurons

- Initial test: 2 hidden layers (32 → 16 neurons) with ReLU → very poor performance (loss ≈ 1.5).
- Adding more neurons/layers and switching to smoother activations significantly improved results.
- Final choice (128 → 128 → 64) gave ~35–40% improvement in convergence.

2. Activation Functions

- **ReLU:** Performed poorly; unsuitable for periodic signals (non-smooth, piecewise linear).
- **SiLU:** Smoother than ReLU, gave ~5% gain but still insufficient.
- **Tanh:** Much better (~20–25% improvement), captured smooth oscillations.
- **SinActivation (custom):** Outperformed all others, ~800% faster convergence and highest accuracy. This matches the intuition that a sine-based activation is naturally aligned with periodic function approximation.

3. Loss Function

- **L1 loss:** Initially tested; emphasized larger errors but overall less effective.
- **L2 loss (MSE):** Gave ~120% improvement compared to L1 and was chosen as the final loss function.

4. Dataset Scaling

- **Original dataset:** poor generalization.
- **2.5× dataset:** achieved very high accuracy with low loss ($\sim 4.2e-7$).
- **5× dataset:** further improved loss ($\sim 1.6e-7$).
- **10× dataset:** marginally lower loss (~7% improvement) but high training time and risk of overfitting. Final choice: 5× dataset.

5. Optimizer & Scheduler

- **Optimizer:** AdamW (lr=0.001, weight_decay=0.01) worked best.
- **Scheduler:** ReduceLROnPlateau (factor=0.5, patience=100) provided stable convergence.

- Other optimizers/schedulers were tested but consistently worse or slower.

Result

I experimented with multiple configurations to identify the most effective setup for approximating the target function. Below are the key findings:

1. Effect of Activation Functions

Function	Observed Loss	Comment
ReLu	~1.8	Performed poorly, failed to capture smooth periodic behavior.
SiLu	~0.3-0.7	Slightly better, smoother than ReLu, but insufficient.
TanH	~0.01-0.02	Captured oscillations better, noticeable improvements.
SinAcitvation	~1.6e-7 – 4.2e-7	Outstanding performance, naturally aligned with periodic signals, fastest convergence.

The custom sinusoidal activation was the clear winner, reducing error by several orders of magnitude compared to standard activations.

2. Effect of Dataset Scaling

Size	Training Time	Best Loss	Comment
Original	~1 min	Poor Convergence	Did not approximate well
2.5x	~2.5 min	~4.2e-7	Very high accuracy, nearly perfect
5x	~3 min	~1.6e-7	Best balance of accuracy and efficiency
10x	~7-8min	Slight improvement (~7%)	Overkill: much longer training, little gain

Training on a dataset 5× larger than the evaluation set gave the best trade-off between accuracy and computational cost. Increasing to 10× offered negligible benefits.

3. Final Performance

With the chosen configuration (3 hidden layers, SinActivation, AdamW optimizer, L2 loss, 5× dataset, 1000 epochs), the network achieved a final loss of $\sim 1.6e-7$.

The predicted curve matches the analytical function almost perfectly across the evaluation domain.

Conclusion

In this lab, I successfully implemented and trained a neural network (SineNet) to approximate the periodic function,

$$f(x) = \sin(2x) + \sin(x)$$

Key takeaways:

- Increasing the network depth and width improved its ability to approximate higher-frequency components.
- The choice of activation function was critical: while ReLU and SiLU underperformed, tanh gave better results, and a custom sinusoidal activation yielded near-perfect approximation.
- Training on a dataset 5× larger than the evaluation set provided the best generalization without excessive training cost.
- Using L2 loss and AdamW with a learning rate scheduler enabled stable convergence.

Overall, the final model achieved an extremely low error ($\sim 1.6e-7$) and closely reproduced the target function. This demonstrates the strength of neural networks as universal function approximators and highlights the importance of careful design choices (architecture, activation, dataset, optimizer) when solving such problems.